

Entrega 5 - Static Test

Ángel David Suescun Romero

asuescunr@unal.edu.co

Laura Camila Perilla Torres

lperillat@unal.edu.co

Juan Pablo Rodríguez Briceño

jrodirguezbr@unal.edu.co

Wilson Franco Martinez

wfrancom@unal.edu.co

Asignatura: Ingeniería de software I

Docente: Oscar Eduardo Alvarez Rodriguez

Universidad Nacional de Colombia

Sede Bogotá

Bogotá D.C

Colombia

TABLA DE CONTENIDOS

Herramientas utilizadas.....	3
Configuración aplicada.....	3
Resultados obtenidos.....	3
ChecStyle:.....	3
PMD:.....	4

ÍNDICE DE FIGURAS

Figura 1: Resultados CheckStyle.....	3
Figura 2: Resultados PMD.....	4
Figura 3: Corrección CheckStyle.....	5
Figura 4: Errores JavaDoc.....	5
Figura 5: Corrección PMD.....	5
Figura 6: Falso Positivo PMD.....	6

Herramientas utilizadas

Se utilizaron las herramientas **Checkstyle** y **PMD** para realizar el análisis estático del código fuente del proyecto. Checkstyle fue empleado para verificar el cumplimiento de estándares de codificación, estilo y formato del código Java, mientras que PMD se utilizó para identificar posibles errores, código duplicado y malas prácticas de programación.

Configuración aplicada

No se realizaron personalizaciones en los archivos de configuración ni se añadieron reglas adicionales durante esta fase del análisis, para la interpretación de los resultados, no se tuvieron en cuenta las advertencias relacionadas con la documentación Javadoc

Resultados obtenidos

CheckStyle:

Se analizaron 56 archivos del proyecto mediante Checkstyle. La herramienta reportó 1388 advertencias y 0 errores. La mayoría de las advertencias corresponden a problemas de indentación (1102 casos).

Checkstyle Results			
The following document contains the results of Checkstyle 9.3 with google_checks.xml ruleset.			
Summary			
Files	Info	Warnings	Errors
56	0	1388	0

Figura 1: Resultados CheckStyle

PMD:

Por su parte, PMD identificó dos advertencias de prioridad media (Priority 3). La primera correspondía a un parámetro de constructor que no era utilizado dentro de la clase, situación que fue corregida para mejorar la limpieza del código. La segunda advertencia señalaba un método privado aparentemente sin uso; sin embargo, tras la revisión se determinó que dicho método estaba asociado al ciclo de vida de una entidad JPA mediante la anotación `@PostLoad`, por lo

que es invocado automáticamente por Hibernate. En consecuencia, este hallazgo fue clasificado como un falso positivo y no requirió modificaciones.

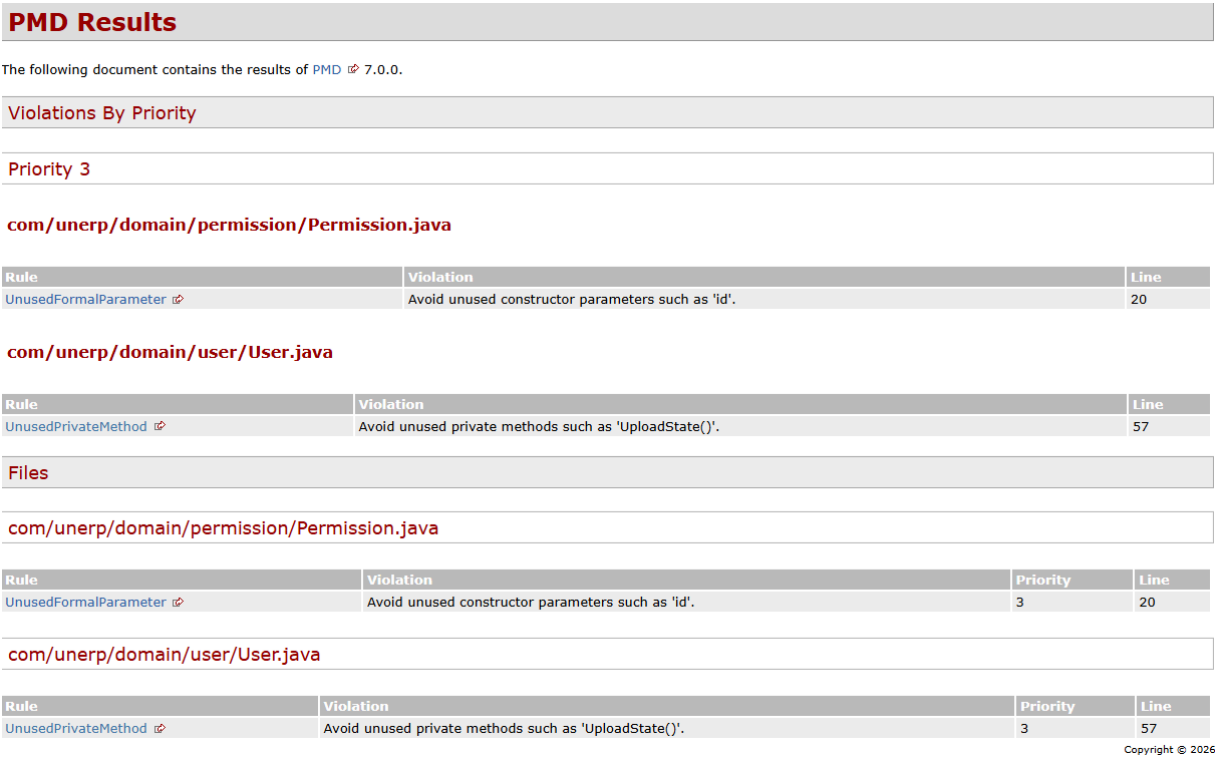


Figura 2: Resultados PMD

Evidencia de corrección

Checkstyle Results

The following document contains the results of [Checkstyle](#) 9.3 with google_checks.xml ruleset.

Summary

Files	Info	Warnings	Errors
56	0	108	0

Figura 3: Corrección CheckStyle

Rules			
Category	Rule	Violations	Severity
javadoc	MissingJavadocMethod - scope: "public" - tokens: "METHOD_DEF, CTOR_DEF, ANNOTATION_FIELD_DEF, COMPACT_CTOR_DEF" - minLineCount: "2" - allowedAnnotations: "Override, Test"	53	Warning
	MissingJavadocType - scope: "protected" - excludeScope: "nothing" - tokens: "CLASS_DEF, INTERFACE_DEF, ENUM_DEF, RECORD_DEF, ANNOTATION_DEF"	55	Warning

Figura 4: Errores JavaDoc

Publicado el: 2026-06-14 | Versión: 0.0.1-SNAPSHOT

Built by 

PMD Results

The following document contains the results of [PMD](#) 7.0.0.

Violations By Priority

Priority 3

com/unerp/domain/user/User.java

Rule	Violation	Line
UnusedPrivateMethod	Avoid unused private methods such as 'uploadState()'.	57

Files

com/unerp/domain/user/User.java

Rule	Violation	Priority	Line
UnusedPrivateMethod	Avoid unused private methods such as 'uploadState()'.	3	57

Copyright © 2026..

Figura 5: Corrección PMD

```
@PostLoad no usages Angel-Suescun
private void uploadState() {
    if ("activo".equalsIgnoreCase(this.stateString)) {
        this.state = new ActiveState();
    } else {
        this.state = new InactiveState();
    }
}
```

Figura 6: Falso Positivo PMD